

SAE 2.02

Comparaison algorithmique d'un problème

BUT Informatique — IUT d'Orléans
Année universitaire 2025–2026

Membres du groupe :

JUSKO Nathaniel
DESSENEAUX-AURIOL Florient
MAKHLOUF Zakaria
BELHAMIDA Adil
BOUSLIM Mourad

GitHub : <https://github.com/NathanielJusk/SAE-2.02-Comparaison-algorithmique-d-un-probl-me>

1. Présentation de l'application

Notre application Java permet de modéliser des livres-jeu sous forme de graphes orientés. Elle permet les fonctionnalités suivantes :

- Création et chargement d'un livre-jeu depuis un fichier CSV
- Génération automatique ou interactive d'un livre
- Recherche d'une solution avec deux algorithmes différents (BFS et Dijkstra)
- Sauvegarde du livre-jeu généré en CSV

Le livre-jeu est modélisé par un graphe orienté pondéré $G = (V, E)$ où chaque sommet représente une page, chaque arc représente un lien possible entre deux pages, et le poids d'un arc correspond au temps de résolution de l'énigme de la page source.

1.1 Architecture générale du projet

L'application est organisée autour de plusieurs classes principales représentées dans le diagramme de classes.

Lors des améliorations du diagramme de classes, plusieurs classes et interfaces ont été ajoutées afin de mieux répartir les responsabilités dans l'application. Une interface `AlgorithmeRecherche` regroupe les algorithmes `AlgorithmeDijkstra` et `AlgorithmeBFS`, tandis qu'une interface `IGenerateur` regroupe `GenerateurV1` et `GenerateurV2` pour distinguer la génération automatique de la génération interactive. La classe `AppLivreJeu` centralise les interactions avec l'utilisateur, alors que `LecteurHistoire` et `SauvegardeLivreJeu` prennent en charge respectivement le chargement du fichier CSV et l'export du graphe en DOT. Ces choix ont permis d'éviter des surcharges inutiles, d'améliorer la lisibilité du projet et de rendre le code plus maintenable.

Classe `LivreJeu`

La classe `LivreJeu` représente la structure principale du projet. Elle contient l'ensemble des pages du livre, la page de départ, la page de sortie ainsi que le graphe représentant les transitions entre les pages.

Elle permet notamment :

- d'ajouter des pages ;
- d'ajouter des liens ;
- d'accéder aux différentes pages ;
- de charger un livre depuis un fichier CSV ;
- de sauvegarder un livre ;
- d'utiliser les algorithmes de recherche.

Cette classe centralise donc la majorité des opérations importantes de l'application.

Classe `Page`

La classe `Page` modélise une page du livre-jeu. Chaque page possède :

- un identifiant ;
- une énigme ;
- un temps de résolution ;
- une liste d'objets récupérables.

Les objets récupérés sont utilisés lors de la recherche de solution afin de vérifier qu'un chemin respecte bien les contraintes du livre.

Classe Objet

La classe Objet représente les objets pouvant être récupérés au cours de l'aventure. Chaque objet possède principalement un nom et peut être défini comme requis pour terminer le livre.

Classe Enigme

La classe Enigme contient le texte de l'énigme ainsi que son temps de résolution. Le temps de résolution est utilisé comme poids dans le graphe, ce qui permet d'appliquer l'algorithme de Dijkstra.

Classe Solution

La classe Solution permet de stocker les résultats retournés par les algorithmes. Elle contient :

- le chemin trouvé ;
- la longueur du chemin ;
- le temps total de parcours ;
- le temps d'exécution de l'algorithme.

Elle facilite également l'affichage des résultats dans la console.

Interface AlgorithmeRecherche

L'interface AlgorithmeRecherche définit une méthode commune utilisée par les différents algorithmes de résolution. Les classes AlgorithmeBFS et AlgorithmeDijkstra implémentent cette interface. Cette organisation permet de modifier facilement l'algorithme utilisé sans changer le reste du programme.

Interface Generateur

L'interface Generateur définit les méthodes utilisées pour générer un livre-jeu. Deux versions sont implémentées :

- GenerateurV1 : génération automatique aléatoire ;
- GenerateurV2 : génération interactive.

Cette séparation rend le projet plus modulaire et facilite l'ajout de nouveaux générateurs.

2. Justification de la correction des algorithmes

2.1 Algorithme BFS

Principe

L'algorithme BFS explore le graphe en largeur d'abord, c'est-à-dire qu'il visite toutes les pages accessibles en 1 étape avant celles accessibles en 2 étapes, etc. On utilise une file pour stocker les chemins en cours d'exploration.

Adaptation au livre-jeu

La contrainte des objets requis complique la recherche : deux passages par la même page peuvent être différents si les objets collectés jusqu'ici sont différents. Pour éviter d'explorer plusieurs fois le même état, on définit un état comme le couple (page courante, ensemble des objets déjà collectés). On ne marque un état visité que si ce couple a déjà été exploré.

Preuve de correction

Terminaison : le nombre d'états possibles est fini (pages et sous-ensembles d'objets). Comme chaque état n'est exploré qu'une seule fois, la boucle se termine nécessairement.

Validité de la solution : lorsqu'un chemin atteint la page de sortie, on vérifie que tous les objets requis ont été collectés sur ce chemin. Si ce n'est pas le cas, l'exploration continue. La solution retournée doit donc toujours satisfaire les quatre conditions : partir du départ, arriver à la sortie, respecter les arcs du graphe, et avoir collecté tous les objets requis.

Optimalité en longueur : BFS garantit de trouver le chemin avec le moins de pages possible parmi toutes les solutions valides. BFS explore les chemins par longueur croissante : le premier chemin valide trouvé est nécessairement le plus court en nombre d'étapes.

2.2 Algorithme de Dijkstra

Principe

L'algorithme de Dijkstra calcule le chemin de coût minimum entre un sommet source et tous les autres sommets d'un graphe pondéré à poids positifs. Ici, le coût d'un arc correspond au temps de résolution de l'énigme de la page source.

Adaptation au livre-jeu

On initialise le coût de la page de départ à 0 et celui de toutes les autres pages à l'infini. À chaque itération, on sélectionne la page non marquée de coût minimum, on la marque, puis on met à jour les coûts de ses voisins si un chemin plus court est trouvé.

Preuve de correction

Terminaison : à chaque itération, une page est retirée de la liste des pages à traiter et marquée. Comme il y a un nombre fini de pages, la boucle finit par se terminer.

Validité du chemin : l'invariant de Dijkstra garantit que lorsqu'une page est marquée, son coût est définitif et minimal. Le chemin reconstruit en remontant les prédécesseurs est donc bien un chemin valide dans le graphe de la page de départ à la page de sortie.

Optimalité en temps de parcours : Dijkstra trouve le chemin dont la somme des temps d'énigmes est minimale. Cela en fait l'algorithme idéal pour minimiser le temps total passé dans le livre-jeu.

Limite connue de notre implémentation

Notre implémentation de Dijkstra optimise uniquement le temps de parcours. Si le chemin optimal en temps ne contient pas tous les objets requis, l'algorithme retourne null sans chercher d'alternative. À l'inverse, BFS gère cette contrainte car il explore tous les états possibles (page + objets collectés). Dans la pratique, pour les livres générés par notre application, les objets requis correspondent à l'ensemble des objets disponibles sur toutes les pages, ce qui signifie que tout chemin valide collecte automatiquement tous les objets. La limite de Dijkstra n'est donc pas visible dans nos tests.

3. Évaluation expérimentale

3.1 Protocole expérimental

Le temps d'exécution est mesuré directement dans le code Java avec `System.currentTimeMillis()` avant et après l'exécution de chaque algorithme. La valeur est stockée dans l'objet `Solution` et affichée par `toString()`.

3.2 Comparaison BFS vs Dijkstra

Tableau des résultats sur 5 livres générés par taille :

Nb Pages	Algo	Longueur solution	Temps parcours (s)	Temps d'exécution (ms)
10	BFS	9 pages	72	10
10	Dijkstra	9	72	2

Analyse

BFS trouve systématiquement la solution avec le moins de pages (chemin le plus court). En contrepartie, le temps de parcours et/ou d'exécution peut être plus élevé car il ne tient pas compte des poids des énigmes.

Dijkstra trouve la solution avec le temps de parcours et/ou d'exécution minimum, c'est-à-dire le chemin dont les énigmes sont globalement les plus rapides à résoudre. Le chemin peut cependant contenir plus de pages que celui de BFS.

Les deux algorithmes trouvent toujours une solution sur nos livres de test quand il en existe une. Le temps d'exécution des deux algorithmes reste très faible pour des livres de taille inférieure à 200 pages, ce qui montre que les deux approches sont adaptées au problème.

3.3 Comparaison GenerateurV1 vs GenerateurV2

Description des deux générateurs

GenerateurV1 (génération aléatoire) : sélectionne aléatoirement n pages parmi celles disponibles dans le CSV, les relie en chaîne linéaire et ajoute des liens retour vers le départ tous les 3 pas. La génération ne nécessite aucune interaction utilisateur.

GenerateurV2 (génération interactive) : l'utilisateur choisit manuellement les pages, les liens entre pages et les objets requis. Cette approche offre plus de contrôle mais nécessite une saisie utilisateur, ce qui la rend inappropriée pour des tests automatisés de performance.

4. Manuel utilisateur

4.1 Prérequis

Avant de compiler et d'exécuter l'application, s'assurer d'avoir installé Java et Maven. Pour vérifier l'installation :

```
java -version  
mvn -version
```

4.2 Compilation avec Maven

Se placer dans le dossier du projet (le dossier contenant le fichier pom.xml) :

```
cd livrejeu
```

Compiler le projet et générer le JAR :

```
mvn clean package -DskipTests
```

Si la compilation réussit, le fichier JAR est généré dans le dossier target/ :
target/livrejeu-1.0-SNAPSHOT.jar

4.3 Exécution de l'application

Depuis le dossier livrejeu/, exécuter la commande suivante :

```
java -cp "target/livrejeu-1.0-SNAPSHOT.jar;target/dependency/*" com.livrejeu.AppLivreJeu
```

Note : l'application charge le fichier CSV de l'histoire depuis un chemin relatif. Il faut donc lancer la commande depuis le dossier livrejeu/ et non depuis target/.

4.4 Utilisation du menu

Au lancement, l'application affiche un menu interactif avec les options suivantes :

1. Générer un livre (aléatoire - V1)
2. Générer un livre (interactif - V2)
3. Afficher le livre
4. Résoudre
5. Sauvegarder (CSV)
0. Quitter

Exemple de session

Voici un exemple de session type pour générer et résoudre un livre de 10 pages :

Votre choix : 1

Nombre de pages : 10

OK : Livre généré avec 10 pages.

Votre choix : 4

1. Dijkstra — chemin le plus RAPIDE
2. BFS — chemin le plus COURT

Votre choix : 2

=== Solution ===

Longueur : 9 pages

Temps parcours : 18 sec

Temps exécution: 2 ms

4.5 Format du fichier CSV

Le fichier histoire.csv situé dans src/main/java/com/livrejeu/histoireCsv/ définit les pages disponibles. Chaque ligne correspond à une page selon le format :

id;texte_enigme;temps_resolution;objet1,objet2,...

Exemple :

1;Vous êtes devant une porte verrouillée...;3;clé

2;Un garde vous barre le passage...;5;

Pour enrichir le livre-jeu, il est possible d'ajouter des lignes dans ce fichier en respectant ce format.

Conclusion

Notre application permet de générer un livre-jeu, de le créer de plusieurs manières et de tester

différents algorithmes sur celui-ci.